DFSha

Sistema de archivos distribuido con alta disponibilidad

Informe técnico · Versión 1.0

Campo	Detalle
Fecha	19 de octubre de 2026
Repositorio	github.com/Valenciavm/DFS (rama **main**)
Opción del proyecto	Opción 1 — Cliente/Servidor (arquitectura Maestro–Trabajadores, estilo HDFS)
Integrantes	Martín Valencia Vallejo Samuel Enrique Rivero Urribarrí Sergio Delúquez González
Curso	ST0263 Tópicos Especiales en Telemática / SI3007 Sistemas Distribuidos, 2026-2 · Proyecto 1
Porcentaje de finalización	**90 %**

Qué falta (≈ 10 %): cifrado TLS/mTLS, cifrado en reposo AES-256-GCM, 2FA TOTP y despliegue en la nube con endurecimiento de seguridad. El despliegue local con Dockerfiles y Docker Compose sí existe, pero es solo de desarrollo.

Cómo leer este informe. Cada capacidad está marcada como **Implementado** (existe en el código y se probó), **Parcial**, **Diseñado** (especificado pero no escrito) o **Pendiente**. Las cifras de la sección 7 se midieron sobre el código real de la rama main.

Resumen de estado

Capacidad	Estado
NameNode: subida, commit, lectura, FS, ACL, locks, GC, re-replicación	Implementado
API REST (auth, archivos, fs, grupos, nodos, admin, ha)	Implementado
Placement ponderado por bloque	Implementado
DataNode gRPC (Write/Read/Stat/Health), pipeline, rangos, tokens, latidos, fencing por término	Implementado
Autenticación Argon2id, JWT, API keys, grupos, ACL	Implementado
Metadatos en PostgreSQL con réplica en caliente	Implementado
Lease, term, segundo NameNode, modo solo-lectura, stepdown	Implementado
Cliente Python y CLI	Implementado (transferencias secuenciales)
Dockerfiles y Docker Compose local (2 NameNode, 5 DataNode, PostgreSQL primario y réplica, cliente)	Implementado (solo desarrollo)
Promoción de réplica PostgreSQL	Manual (script)
TLS / mTLS	**Pendiente**
Cifrado en reposo AES-256-GCM	**Pendiente**
2FA TOTP	**Pendiente**
Despliegue en nube (AWS/GCP)	**Pendiente**
Plan de pruebas T1–T24 y mediciones de rendimiento	**Pendiente**
Asignación incremental de bloques	Diseñado

1. Objetivo y marco teórico

1.1 Objetivo

Diseñar, implementar y evaluar un sistema de archivos distribuido basado en bloques que almacene archivos grandes sobre varias máquinas, de modo que la caída de un nodo no cause pérdida de datos ni interrupción del servicio, con acceso controlado. Ofrece una vista jerárquica tipo Linux (ls, cd, mkdir, put, get…) y oculta dónde está físicamente cada bloque.

Objetivos específicos:

- Particionar cada archivo en bloques y distribuirlos entre nodos (los archivos se reparten entre nodos y cada archivo entre varios nodos).
- Replicar cada bloque (factor 3) y reparar automáticamente las réplicas perdidas.
- Consistencia fuerte de metadatos y visibilidad atómica de los archivos (modelo WORM).
- Alta disponibilidad del plano de control con dos NameNodes y PostgreSQL con réplica.
- Controlar el acceso (JWT, API keys, ACL por usuario y grupo). El cifrado de canales y de almacenamiento y el 2FA quedaron pendientes.
- Evaluar el sistema con pruebas automáticas y un escenario de demostración reproducible.

1.2 Marco teórico breve

Sistemas de archivos distribuidos. Presentan como un único espacio de nombres un almacenamiento que vive en muchas máquinas. Se buscan transparencia, escalabilidad, tolerancia a fallos y seguridad.

Arquitectura GFS/HDFS. Ghemawat et al. (2003) y Shvachko et al. (2010) separan el plano de control del plano de datos: un nodo maestro (NameNode) guarda el espacio de nombres y el mapa archivo → bloques → réplicas, y muchos DataNodes almacenan los bloques. El cliente consulta al maestro solo por metadatos y mueve los bytes directamente con los DataNodes. DFSha adopta este modelo.

Bloques frente a objetos. Un almacenamiento por bloques parte los archivos en fragmentos de tamaño fijo (64 MiB por defecto), lo que permite paralelizar, repartir carga y reparar a granularidad fina. El almacenamiento de objetos (estilo S3) trata el archivo como una unidad.

Replicación y consistencia. Cada bloque se copia en varios nodos mediante un pipeline. Con datos inmutables (WORM) desaparecen los conflictos de actualización. Los metadatos requieren consistencia fuerte, que DFSha obtiene con un único líder que escribe en una base transaccional. Se separa el *estado duro* (árbol, permisos, bloques) del *estado blando* (nodos vivos y ubicación de réplicas), que se reconstruye con latidos e informes de bloques.

Alta disponibilidad. Elección de líder por *lease* (arrendamiento con vencimiento) sobre PostgreSQL y un número de término (*term*) como fencing: los DataNodes ignoran órdenes de términos anteriores, lo que neutraliza a un líder obsoleto. Los fallos se detectan con latidos.

Seguridad. Implementado: contraseñas con Argon2id, JWT, API keys, ACL por usuario o grupo y tokens de bloque firmados con HMAC que el DataNode valida localmente. Diseñado pero pendiente: TLS/mTLS, cifrado en reposo AES-256-GCM y TOTP (RFC 6238).

2. Descripción del servicio y problema abordado

Un solo servidor de archivos es un punto único de falla y un límite de capacidad. Cuando el volumen supera lo que cabe en una máquina, y se exige que el servicio no se interrumpa ni se pierdan archivos ante la caída de nodos, hace falta distribuir y replicar el almacenamiento manteniendo control de acceso.

DFSha es un sistema de archivos distribuido por bloques al estilo HDFS:

- Los archivos se dividen en bloques de tamaño fijo repartidos entre DataNodes, y cada bloque se replica (factor 3).
- Un NameNode mantiene todo el metadato: espacio de nombres, permisos, bloques de cada archivo y ubicación de réplicas.
- Un cliente (CLI y biblioteca Python) ofrece la vista jerárquica y oculta la ubicación de los datos.

- Los archivos son WORM: un archivo confirmado es inmutable; modificarlo equivale a subir una nueva versión (*overwrite*).

2.1 Mapa de requisitos del enunciado

Requisito	Cómo se cubre	Estado
RF1 Gestión del FS (ls, cd, mkdir, rmdir, rm...)	Endpoints /v1/fs/* y cliente. *cd* es estado del cliente validado con *stat*	Implementado
RF2 Transferencia (send/receive)	*put_file* y *get_file* (alias send/receive) con plan del NameNode y gRPC a los DataNodes	Implementado
RF3 Acceso (open/close/read/lock)	Lecturas por rango (offset/length) y locks shared/exclusive con expiración. Sin *write* en sitio por ser WORM	Parcial
Archivos y cada archivo repartidos entre nodos	Colocación por bloque con muestreo ponderado (sección 5)	Implementado
Alta disponibilidad, replicación y consistencia	Lease en PostgreSQL, 2 NameNodes, fencing, quórum de commit, re-replicación	Implementado
Seguridad (tránsito, reposo, acceso, entre nodos)	JWT, API keys, ACL, grupos y tokens de bloque implementados. Faltan TLS/mTLS, AES-GCM y 2FA	Parcial
Transparencia	El cliente nunca elige nodos; el NameNode decide	Implementado
Cada nodo expone una API	NameNode: REST con OpenAPI (/docs). DataNode: gRPC BlockService	Implementado

Opción asignada: Opción 1 — Cliente/Servidor. El servidor se implementa por composición y distribución (NameNodes + DataNodes + base de datos de metadatos) dentro de una red privada, con APIs abiertas hacia el cliente: REST para metadatos y gRPC para bloques.

3. Arquitectura del sistema y diagramas

3.1 Visión general

DFSha es una arquitectura **Maestro–Trabajadores**. El **plano de control** (2 NameNodes con PostgreSQL primario y réplica) decide dónde va cada bloque. El **plano de datos** (5 DataNodes) almacena y transmite los bytes. El NameNode atiende únicamente REST; los DataNodes atienden únicamente gRPC para bloques y son quienes llaman al NameNode (registro, latidos, informes), de modo que el NameNode nunca necesita alcanzar a un DataNode.

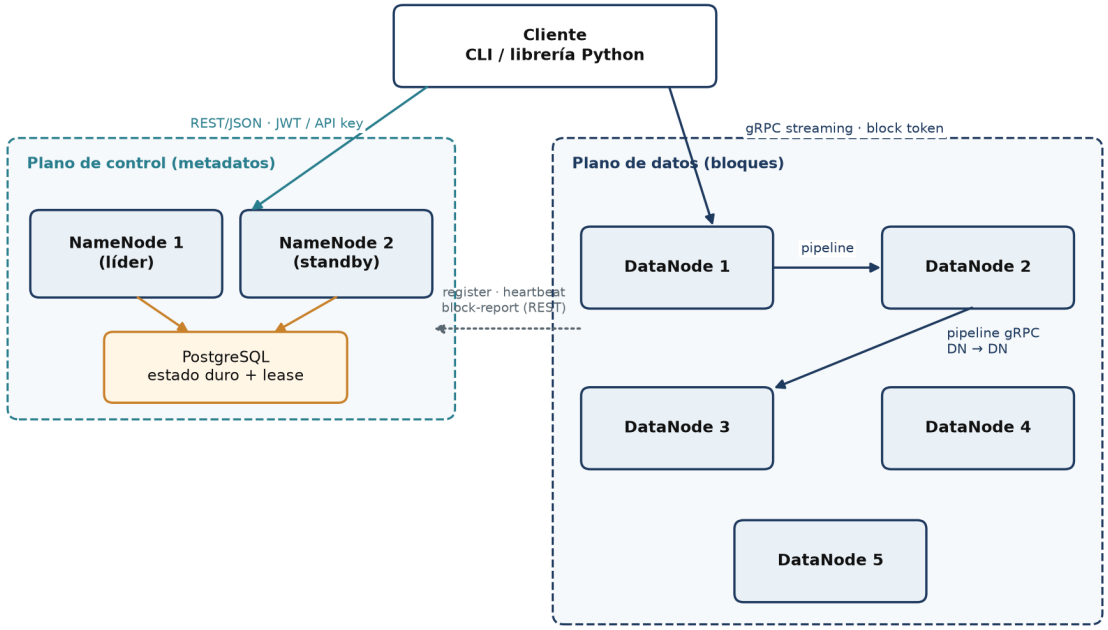


Figura 1. Arquitectura general: plano de control (2 NameNodes + PostgreSQL) y plano de datos (5 DataNodes).

Por qué REST para metadatos y gRPC para bloques. Los mensajes de control son pequeños y poco frecuentes, así que REST da depuración (curl) y documentación automática (OpenAPI). Los bloques usan gRPC (HTTP/2 + Protocol Buffers), que permite transmitir en *chunks* y reenviar al siguiente nodo mientras se recibe, con un contrato estricto desde el archivo .proto.

3.2 Componentes

Componente	Responsabilidad	Código
NameNode	Espacio de nombres, permisos, mapa archivo→bloques→réplicas, colocación, re-replicación, GC, tokens, autenticación, lease	server/nodes/namenode/
DataNode	Almacenar y servir bloques por gRPC, pipeline, integridad, registro, latidos e informes	server/nodes/datanode/
Cliente	Biblioteca DFShaClient y CLI. Divide en bloques, verifica checksums, reintenta contra el líder (≈1 030 líneas)	client/
Metadatos	Interfaz MetadataStore con implementación en memoria (pruebas) y PostgreSQL (real)	server/nodes/namenode/store/
Común	Configuración, errores REST↔gRPC, tokens HMAC, rutas, reloj simulable	server/common/

3.3 Flujos principales

Escritura (put). (1) El cliente pide subir un archivo; el NameNode valida permisos, reserva la ruta (archivo PENDING, invisible), calcula los bloques y un pipeline de DataNodes por bloque, y devuelve tokens de escritura. (2) El cliente transmite cada bloque por gRPC al primer DataNode del pipeline; este lo reenvía al siguiente a medida que llega, cada nodo verifica el SHA-256, lo guarda de forma atómica y avisa *block-received*. (3) El cliente llama a *commit*; el NameNode comprueba que cada bloque tenga al

menos MIN_REPLICAS_COMMIT réplicas confirmadas y checksums coincidentes, y publica el archivo (COMMITTED) en una sola operación.

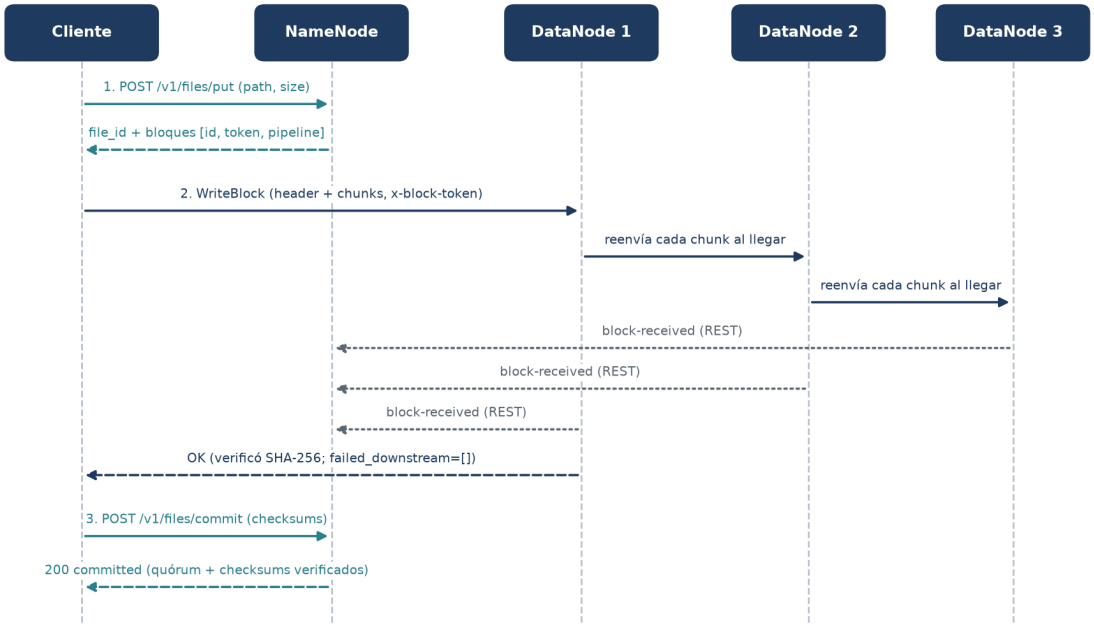


Figura 2. Secuencia de escritura de un bloque a lo largo del pipeline y confirmación (commit).

Lectura (get). El NameNode devuelve bloques, checksums, tokens de lectura y las réplicas vivas. El cliente descarga cada bloque, cae a otra réplica si falla, verifica el checksum y concatena por índice. Las lecturas parciales usan offset/length.

Recuperación ante fallos. Un DataNode sin latidos por DEAD_NODE_TIMEOUT_S (30 s) se marca muerto. Un lazo de fondo busca bloques con menos réplicas vivas que el objetivo y envía órdenes *replicate* en la respuesta al latido del nodo que tiene copia; el destino confirma con *block-received* hasta restaurar el factor.

3.4 Interior del NameNode

El núcleo (Namenode) es una clase de Python puro, sin código web, que se prueba con reloj simulado. La capa FastAPI es un adaptador delgado que extrae la identidad, llama al núcleo y traduce DfshaError a HTTP. Cuatro tareas de fondo mantienen el cluster sano, y las cuatro están implementadas.

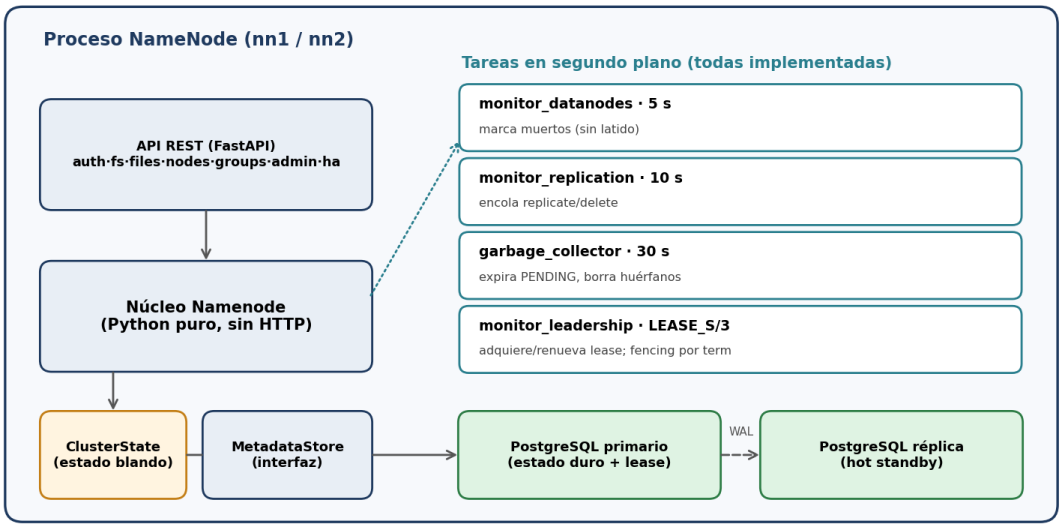


Figura 3. Componentes del proceso NameNode, con almacenamiento en PostgreSQL (primario y réplica) y las cuatro tareas de fondo, incluido el lease.

3.5 Modelo de metadatos y estado

	Estado duro	Estado blando
Contenido	Usuarios, claves de API, grupos, árbol de directorios, archivos y bloques, ACL, locks, lease del líder, auditoría	DataNodes vivos, espacio libre, ubicación de cada réplica, comandos pendientes, pipelines asignados
Dónde vive	PostgreSQL (visible para ambos NameNodes). Implementación en memoria solo para pruebas	Memoria del líder (ClusterState)
¿Se pierde al reiniciar?	No	Sí, y se reconstruye con registro, latidos e informes de bloques

Entidades principales: **FileMetadata** (id UUID4, propietario, tamaño, estado PENDING | COMMITTED | ABORTED, bloques ordenados), **BlockMetadata** (id opaco, índice, tamaño, SHA-256), **DirectoryMetadata** y **DataNodeInfo**. El nombre completo de una ruta se calcula y no se almacena en cada nodo, así que *mv* cambia una sola referencia. La lista de réplicas contiene solo copias confirmadas; el pipeline es únicamente el plan.

3.6 Alta disponibilidad del NameNode (implementada)

- Una fila singleton **leader(holder, url, term, lease_expires_at)** en PostgreSQL decide el líder.
- *acquire_leader* es un UPDATE atómico: solo adquiere un lease vencido o renueva el propio, e incrementa el *term* en cada cambio de líder. *renew_leader* extiende el lease solo si holder y term coinciden. *release_leader* lo expira (usado por POST /v1/ha/stepdown, solo admin).
- Las escrituras de estado duro verifican el lease dentro de la misma transacción (fencing en la base de datos). Un NameNode que no es líder responde **503 NOT_LEADER** con *leader_url*; un middleware rechaza todo salvo /health y /v1/ha/status.
- Tras un cambio de líder hay un periodo de gracia (FAILOVER_GRACE_S) mientras llegan los informes de bloques. Existe un modo degradado (solo-lectura) cuando la base de datos no está disponible.
- El cliente acepta varias URL de NameNode (NAMENODE_URLS) y, ante NOT_LEADER, reintenta contra *leader_url*.
- **Limitación:** la promoción de la réplica PostgreSQL es manual (scripts/promote-postgres-replica). Hay que detener ambos NameNodes, promover y actualizar POSTGRES_PRIMARY_DSN. No hay failover automático de la base de datos.

4. Especificación de protocolos y APIs

El contrato oficial entre componentes es **docs/API.md (v0.10)**. Si una implementación difiere, el contrato gana hasta acordar el cambio. El OpenAPI de FastAPI está en /docs.

4.1 Canales de comunicación

Interacción	Protocolo	Autenticación
Cliente ↔ NameNode	REST/JSON (HTTP)	JWT o API key
Cliente ↔ DataNode	gRPC (HTTP/2 + Protobuf), bytes en chunks	Token de bloque en metadato x-block-token
DataNode ↔ DataNode	gRPC (pipeline y re-replicación)	Token de bloque
DataNode → NameNode	REST/JSON (registro, latidos, informes)	Cabecera X-Node-Key
NameNode ↔ BD	SQL (psycopg) sobre red privada	Credenciales

Importante: hoy los canales **no están cifrados** (HTTP y gRPC inseguro). TLS/mTLS está pendiente.

4.2 Convenciones generales

- JSON UTF-8 en REST; bloques como bytes crudos en chunks por gRPC. Fechas ISO-8601 UTC; tamaños en bytes.
- Rutas absolutas en un árbol global; el hogar de cada usuario es /users/<usuario>. El servidor nunca reescribe rutas.
- Checksums SHA-256 sobre los bytes en claro del bloque. file_id es UUID4; block_id es opaco.
- Trazabilidad con X-Request-Id. Los receptores ignoran campos desconocidos.

4.3 API REST del NameNode (prefijo /v1)

Rutas verificadas en server/nodes/namenode/api/routes/.

Grupo	Endpoints	Estado
Autenticación	POST /auth/register, /auth/login; POST|GET /auth/apikeys; DELETE /auth/apikeys/{id}	Implementado
2FA (TOTP)	/auth/2fa/*, /auth/login/2fa. El login devuelve la rama mfa_required, pero la verificación está diferida	**Pendiente**
Archivos	POST /files/put, /files/commit, /files/abort (también /files/{file_id}/commit|abort); GET /files/get; POST /files/tokens	Implementado
Sistema de archivos	GET /fs/ls, /fs/stat; POST /fs/mkdir, /fs/mv; DELETE /fs/rm, /fs/rmdir; GET|PUT /fs/acl; POST /fs/lock, /fs/lock/{id}/renew; DELETE /fs/lock/{id}	Implementado
Grupos	POST /groups; GET /groups/{name}; PUT /groups/{name}/members	Implementado
Nodos (DataNode→NN)	POST /nodes/register, /heartbeat, /block-report, /block-received, /block-corrupt	Implementado
Admin y HA	GET /health, GET /v1/admin/stats, GET /v1/ha/status, POST /v1/ha/stepdown	Implementado

Ejemplo — subida (put):

```
POST /v1/files/put
{ "path": "/users/ana/report.pdf", "size": 10485760, "overwrite": false }
201 Created
{ "file_id": "7b0c9e0e-...", "block_size": 4194304, "replication": 3,
  "blocks": [ { "index": 0, "block_id": "blk_7b0c9e0e_000000", "size": 4194304,
                "token": "<block-token-write>",
                "pipeline": [ {"node_id":"dn1","addr":"dn1:9000"}, ... ] } ] }
```

Formato de error (único en toda la API): `{ "error": { "code": "FILE_NOT_FOUND", "message": "..." } }`. Códigos principales: 400 BAD_REQUEST; 401 UNAUTHORIZED; 403 FORBIDDEN; 404 FILE_NOT_FOUND; 409 ALREADY_EXISTS, UPLOAD_IN_PROGRESS, INSUFFICIENT_REPLICAS, NODE_NOT_REGISTERED; 422 CHECKSUM_MISMATCH; 423 LOCKED; 503 NOT_ENOUGH_DATANODES, NOT_LEADER. Los mismos nombres viajan por gRPC en el metadato final x-dfsha-error-code.

4.4 API gRPC de bloques (BlockService)

Definida en proto/block_service.proto y expuesta por cada DataNode.

RPC	Tipo	Descripción
WriteBlock	client-streaming	Primer mensaje: cabecera (block_id, size, checksum, pipeline); siguientes: chunks. Reenvía al siguiente nodo mientras recibe; verifica SHA-256 y renombra .tmp → .dat de forma atómica. Idempotente. Devuelve failed_downstream
ReadBlock	server-streaming	Lectura por rangos (offset, length) con cabecera de tamaño y checksum del bloque completo; fuera de rango → OUT_OF_RANGE
StatBlock	unario	Tamaño y checksum sin transferir bytes
Health	unario	Estado, espacio libre y número de bloques; no requiere token

Los chunks son de hasta BLOCK_CHUNK_SIZE (1 MiB); un DataNode rechaza chunks de más de 2 MiB.

4.5 Token de bloque

Token = base64url(payload) + "." + base64url(HMAC-SHA256(secreto, payload)) con payload {bid, op (r|w), usr, exp}. Un interceptor gRPC del DataNode valida localmente la firma, la expiración, que bid coincida con el bloque solicitado y que op corresponda al RPC. Un token de un bloque no sirve para otro. Vigencia por defecto: 900 s.

4.6 Control DataNode → NameNode

Llamada	Frecuencia	Contenido / respuesta
register	Al arrancar	node_id, direcciones, capacidad → intervalos y term
heartbeat	Cada 5 s	Espacio libre/usado → { term, commands } (replicate, delete)
block-report	Al registrarse y cada 60 s	Inventario completo → órdenes de borrado de huérfanos
block-received	Al almacenar un bloque	Confirma la réplica (base del quórum de commit)
block-corrupt	Al detectar corrupción	El NameNode descarta la réplica y agenda otra

Fencing por término. Cada respuesta lleva el term del líder. El DataNode recuerda el mayor visto e ignora órdenes de un término menor (líder zombi); al ver un término mayor envía de inmediato un informe completo de bloques. Un latido de un nodo desconocido responde 409 NODE_NOT_REGISTERED y el DataNode se registra de nuevo.

4.7 Parámetros configurables

Variable	Defecto	Función
BLOCK_SIZE	64 MiB (demo: 1 MiB)	Tamaño de bloque
REPLICATION_FACTOR / MIN_REPLICAS_COMMIT	3 / 2	Réplicas objetivo / quórum de commit
HEARTBEAT_INTERVAL_S / DEAD_NODE_TIMEOUT_S	5 / 30	Latido / tiempo para declarar muerto un nodo
BLOCK_REPORT_INTERVAL_S	60	Periodo del informe de bloques
TOKEN_TTL_S / JWT_TTL_S	900 / 3600	Vigencia de tokens de bloque / de acceso
UPLOAD_TTL_S	3600	Una subida sin commit se recolecta pasado este tiempo
LEASE_S / FAILOVER_GRACE_S	15 / 30	Vigencia del lease / gracia tras cambio de líder

5. Algoritmos de particionamiento y distribución

5.1 División de un archivo en bloques

num_bloques = max(1, ceil(tamaño / BLOCK_SIZE)). Todos los bloques miden BLOCK_SIZE excepto el último, que contiene el resto; un archivo vacío genera un bloque de 0 bytes.

5.2 Colocación por bloque (placement)

Implementada en server/nodes/namenode/placement.py como funciones puras con la fuente aleatoria inyectada (reproducible con semilla). Para **cada bloque por separado**:

- **Candidatos:** nodos vivos cuyo espacio utilizable (libre − reservado) cabe el bloque. Si hay menos que REPLICATION_FACTOR → NOT_ENOUGH_DATANODES.
- **Muestreo ponderado sin reemplazo** de REPLICATION_FACTOR nodos distintos, con probabilidad proporcional al espacio utilizable (Efraimidis–Spirakis, calculado como log(u)/w para evitar empates numéricos con pesos grandes).
- **Orden del pipeline:** el nodo con más espacio utilizable va primero.
- **Reserva:** se suma el tamaño del bloque a reserved_bytes de cada nodo elegido para que los bloques siguientes vean el espacio ya comprometido.

El espacio libre favorece a los nodos más vacíos y el azar hace que bloques consecutivos de un mismo archivo caigan en **conjuntos distintos de nodos**: eso es lo que *particiona* el archivo en lugar de duplicarlo entero en todos.

5.3 ¿Por qué se necesitan más DataNodes que el factor de replicación?

El enunciado exige que los archivos se repartan entre nodos y que *cada archivo* se reparta entre varios nodos. Con exactamente REPLICATION_FACTOR nodos, cada bloque debe ir a todos los nodos y cada DataNode termina con una copia completa de cada archivo: particionar es indistinguible de espejar. Además la re-replicación se vuelve imposible, porque reparar un bloque exige un nodo vivo que no lo tenga. Por eso el despliegue mínimo es de **5 DataNodes**.

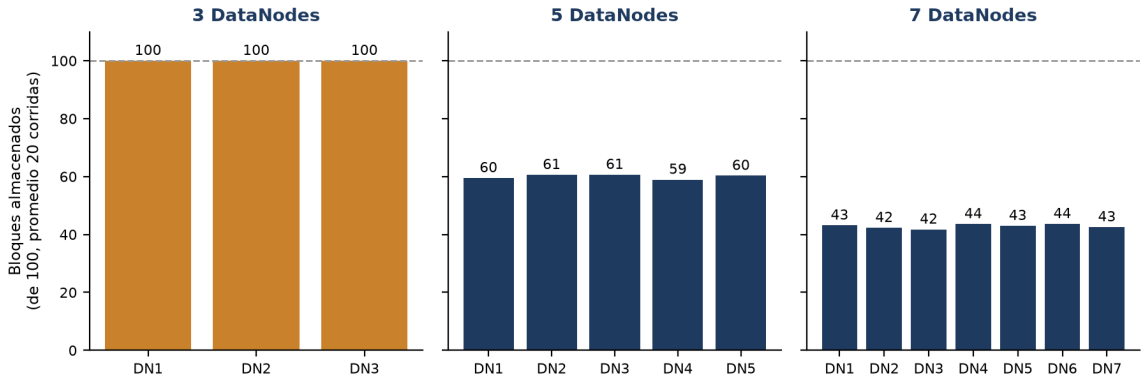


Figura 4. Bloques por DataNode para un archivo de 100 bloques (RF = 3), promedio de 20 corridas con el placement real. Con 3 nodos cada uno guarda 100/100; con 5, ≈ 60 %; con 7, ≈ 43 %.

5.4 Asignación versus confirmación

El pipeline es solo el plan. Un nodo cuenta como réplica cuando confirma (block-received) o cuando lo reporta en un informe de bloques. El quórum de escritura se comprueba siempre contra réplicas confirmadas, nunca contra el pipeline.

5.5 Tolerancia a un nodo caído en el pipeline

Un pipeline es una cadena: un nodo caído en el medio cortaría a todos los de atrás. Por eso un DataNode que no alcanza a pipeline[0] prueba los siguientes en orden y reenvía al primero que responda, pasándole el resto de la cadena; si el reenvío falla a mitad de la transferencia, reproduce la copia local desde disco. Las direcciones omitidas se devuelven en failed_downstream. Tres réplicas degradan a dos, el quórum se cumple y la re-replicación restaura la tercera. Cubierto por pruebas de integración.

5.6 Re-replicación

- Solo se examinan bloques de archivos COMMITTED.
- Prioridad: primero los bloques con menos réplicas vivas.
- Origen: nodo vivo que tiene el bloque. Destino: nodo vivo con espacio que no lo tiene.
- La orden viaja en el latido del nodo origen; no se re-encola el mismo par (bloque, destino) hasta que expire el intento.
- Las réplicas sobrantes (por ejemplo, un nodo que vuelve) reciben una orden delete.

5.7 Asignación incremental de bloques (diseñada, no implementada)

Hoy el put asigna todos los bloques por adelantado, y existe MAX_FILE_SIZE_BYTES (10 GiB) como límite. El diseño propone asignar por lotes de 8–16 bloques con tokens y pipelines frescos, lo que exige un cambio de contrato.

6. Entorno de ejecución (nativo y Docker)

6.1 Ejecución nativa

Cada nodo corre como proceso Python desde la raíz del repositorio. Se probó con Python 3.12.3 y las dependencias de requirements.txt con versiones exactas (FastAPI, Uvicorn, Pydantic, psycopg 3, argon2-cffi, PyJWT, pyotp, qrcode, httpx, grpcio, protobuf). La configuración se hace solo con variables de entorno, sin secretos en el repositorio.

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=. proto/block_service.proto
python -m pytest -q
```

6.2 Docker y Docker Compose (implementado, solo desarrollo)

El repositorio incluye los Dockerfiles (docker/namenode, datanode, client y postgres-replica) y un docker-compose.yml que levanta una red privada con:

- **postgres-primary** y **postgres-replica** (streaming hacia un hot standby, con volúmenes propios).
- **namenode1** y **namenode2** (puertos 8001 y 8002 del host).
- **datanode1 … datanode5**, cada uno con su volumen y direcciones anunciadas por DNS de Compose.
- Un contenedor **client** con las dependencias instaladas.

El perfil de demostración usa bloques de 1 MiB, factor de replicación 3 y quórum de commit 2.

```
cp .env.example .env
docker compose up --build -d
docker compose ps
docker compose exec -T client python scripts/smoke_test.py
docker compose exec client python -m client.cli --username demo-admin ls /
docker compose stop datanode1        # demo: caída de un DataNode
docker compose start datanode1
```

El smoke test inicia sesión, sube un archivo mayor a un bloque, lo descarga y compara bytes y SHA-256. La promoción manual de la réplica PostgreSQL se hace con scripts/promote-postgres-replica.sh (o .ps1) con ambos NameNodes detenidos, y luego se actualiza POSTGRES_PRIMARY_DSN.

Advertencia (del propio README del repositorio): este stack usa HTTP, gRPC inseguro, credenciales de demostración y no cifra los archivos en reposo. No debe exponerse a redes no confiables ni considerarse un despliegue de producción.

6.3 Despliegue en la nube (pendiente)

La topología propuesta es una VM por NameNode, cinco VM para los DataNodes y una base PostgreSQL administrada, con grupos de seguridad que expongan solo el puerto REST de los NameNodes y el gRPC de los DataNodes. No está desplegada ni validada en AWS/GCP.

7. Pruebas y análisis de resultados

Las cifras de 7.1 se obtuvieron ejecutando el código de la rama main en un entorno limpio (Python 3.12.3). Los datos de 7.2 y 7.3 provienen de las pruebas de integración y de la simulación del borrador anterior.

7.1 Suite automática

Resultado de pytest: **429 pruebas aprobadas, 55 omitidas, 0 fallos** (≈ 26 s). Las 55 omitidas son las pruebas contra PostgreSQL real (test_store y test_postgres_store), que requieren la variable POSTGRES_TEST_DSN y no se ejecutaron en esta revisión. Las pruebas unitarias del cliente (unittest) pasan: 8 de 8.

Área	Qué verifica
Núcleo NameNode (subida, lectura, FS, registro, replicación, informes, GC, stats)	Reserva de ruta, quórum y checksums en commit, overwrite, locks, mv, re-replicación, recolección

Área	Qué verifica
Almacén de metadatos	Contrato de MetadataStore (árbol, índices, estados)
Autenticación y ACL	Contraseñas, JWT, API keys, ACL por usuario y grupo
API REST y HA	Casos de extremo a extremo con TestClient; adquisición del lease (test_ha)
Común	Configuración, errores REST↔gRPC, tokens HMAC, rutas, reloj
Plano de datos (gRPC)	Pipeline multi-nodo, checksum erróneo, nodo caído en cada posición, rangos, interceptor de tokens, concurrencia, corrupción
DataNode (lazo de control y almacenamiento)	Registro, latidos, informes, almacenamiento en disco

Cobertura de líneas (solo código de producción): **69,7 %** (2 975 de 4 267 sentencias). Baja frente al 84,9 % del borrador porque el código ahora incluye PostgreSQL y el cliente, y las pruebas de PostgreSQL estaban omitidas.

Módulo	Cobertura
placement.py, routes/groups.py, routes/nodes.py, schemas.py	100 %
Núcleo NameNode (core.py)	81 %
datanode/block_service.py	82 %
datanode/core.py	64 %
store/postgres.py y postgres_auth_store.py	18 % (pruebas omitidas por falta de DSN)
client/client.py	55 %
client/cli.py	0 %

7.2 Escenarios de fallo en el plano de datos

Escenario	Resultado verificado
Bloque replicado a lo largo del pipeline	Idéntico en todos los nodos de la cadena
Checksum incorrecto	Rechazado (INVALID_ARGUMENT); no queda ningún archivo
Nodo posterior caído	Se reporta en failed_downstream; la copia local sobrevive
Nodo del medio caído	Se omite y el bloque llega al siguiente nodo vivo
Lectura por rangos	Se sirven exactamente los bytes pedidos; rangos inválidos → OUT_OF_RANGE
Token ausente, inválido, expirado o de otro bloque	Rechazado por el interceptor
Escritura duplicada y concurrente	Idempotente; un solo bloque válido
Bloque corrupto en disco	DATA_LOSS con metadatos de error

7.3 Evidencia de particionamiento

Se ejecutó 20 veces (semillas 0–19) choose_pipeline del repositorio para un archivo de 100 bloques, factor 3 y nodos de 100 GiB (simulación del borrador, no repetida en esta revisión).

DataNodes	Fracción del archivo por nodo	¿Algún nodo con el archivo completo?	¿Re-replicación posible?
3	100 %	Sí (espejo)	No
5	≈ 60 % (3/5)	No	Sí
7	≈ 43 % (3/7)	No	Sí

El reparto sigue la expectativa teórica RF/N, con diferencias de uno o dos puntos porcentuales entre nodos. Con 3 nodos el sistema refleja en lugar de particionar y no puede reparar réplicas, lo que justifica el mínimo de 5 DataNodes.

7.4 Pruebas de sistema

Existe un smoke test de extremo a extremo (scripts/smoke_test.py) que valida subida, descarga y SHA-256 sobre el cluster Compose. El plan completo de pruebas T1–T24 (caída del líder, re-replicación medida, corrupción en el cluster, concurrencia, seguridad) **no se ejecutó formalmente** y queda pendiente. La demostración de caída de un DataNode y de failover del NameNode se hace de forma manual con Docker Compose, sin métricas registradas.

7.5 Mediciones de desempeño

No se realizaron mediciones de caudal, tiempos de re-replicación ni de failover. Además, el cliente actual transfiere los bloques de forma **secuencial**. El enunciado pide concurrencia y escritura y lectura distribuidas: la distribución entre nodos se cumple en la arquitectura (los bloques se reparten entre DataNodes), pero la paralelización de transferencias en el cliente queda como trabajo pendiente.

8. Estado de implementación y trabajo pendiente

Porcentaje de finalización: 90 %. Lo implementado cubre el núcleo del sistema (almacenamiento distribuido por bloques, replicación, alta disponibilidad con lease, cliente y despliegue local). Lo pendiente es principalmente la capa de cifrado y el despliegue endurecido en la nube.

8.1 Trabajo realizado

- NameNode completo: subida, commit con quórum, lectura, FS, ACL, locks, GC y re-replicación.
- API REST con autenticación (Argon2id, JWT, API keys), grupos y ACL.
- DataNode gRPC con pipeline, lecturas por rango, tokens de bloque, latidos, informes y fencing por término.
- Metadatos en PostgreSQL con réplica en caliente.
- Alta disponibilidad: lease, term, segundo NameNode, modo solo-lectura y stepdown.
- Cliente Python y CLI con reintento hacia el líder.
- Dockerfiles y Docker Compose local con 2 NameNodes, 5 DataNodes y PostgreSQL primario y réplica, más smoke test.
- Pruebas automáticas: 429 aprobadas, cobertura de producción 69,7 %.

8.2 Trabajo pendiente (≈ 10 %)

Pendiente	Detalle
TLS / mTLS	Todos los canales (cliente, DataNode, NameNode, base de datos) van sin cifrar
Cifrado en reposo AES-256-GCM	Los bloques se guardan en claro en los DataNodes
2FA TOTP	Falta la verificación y los endpoints /auth/2fa/*; el login ya devuelve la rama mfa_required
Despliegue en nube y endurecimiento	Solo existe el Compose local de desarrollo; faltan grupos de seguridad y configuración en AWS/GCP
Failover automático de PostgreSQL	La promoción de la réplica es manual
Plan T1–T24 y mediciones de rendimiento	No ejecutados; solo hay smoke test y pruebas automáticas
Transferencias paralelas en el cliente	Hoy los bloques se transfieren de forma secuencial
Asignación incremental de bloques	Solo diseñada
Pruebas PostgreSQL en la suite	55 pruebas se omiten sin POSTGRES_TEST_DSN

8.3 Observaciones sobre el contrato

El contrato vigente es la v0.10. Los endpoints de commit y abort existen en ambas formas (/files/commit y /files/{file_id}/commit). Las diferencias señaladas en el borrador anterior (formato de la renovación de tokens, file_checksum, campos de la respuesta de get y de abort) no se volvieron a verificar en esta revisión y conviene confirmarlas contra docs/API.md antes de la entrega.

8.4 Limitaciones declaradas del diseño

- El NameNode líder es el único escritor de metadatos (cuello de botella a muy gran escala).
- Si se pierden ambos nodos de la base de datos, el servicio queda en solo-lectura.
- Tras un failover hay una ventana corta en la que commit puede requerir reintento.
- Solo WORM: no hay actualizaciones parciales; modificar equivale a subir una nueva versión.
- No hay montaje POSIX (FUSE): la transparencia de acceso es dinámica, mediante CLI y biblioteca.
- gRPC no se puede invocar desde un navegador ni con curl.

Referencias

- Ghemawat, S., Gobioff, H. y Leung, S.-T. *The Google File System*. ACM SOSP, 2003.
- Shvachko, K., Kuang, H., Radia, S. y Chansler, R. *The Hadoop Distributed File System*. IEEE MSST, 2010.

- M'Raihi, D. et al. *TOTP: Time-Based One-Time Password Algorithm*. RFC 6238, IETF, 2011.
- Efraimidis, P. S. y Spirakis, P. G. *Weighted random sampling with a reservoir*. Information Processing Letters, 97(5), 2006.
- Documentación del repositorio: docs/API.md, ARCHITECTURE.md, NAMENODE_MODELS.md, GRPC_BLOCK_SERVICE.md, DATANODE.md, POSTGRESQL.md, REST_API.md.
- gRPC Authors. *gRPC Documentation* y Protocol Buffers Language Guide (proto3).